Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
GRADUATION THESIS

Разработка лабораторного программного комплекса для студентов по определению параметров истечения идеального газа при дросселировании

Обучающийся / Student Жуков Андрей Дмитриевич
Факультет/институт/кластер/ Faculty/Institute/Cluster образовательный центр "Энергоэффективные инженерные системы"
Группа/Group W34601
Направление подготовки/ Subject area 14.03.01 Ядерная энергетика и теплофизика
Образовательная программа / Educational program Информационные технологии в энергетике 2020
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Руководитель ВКР/ Thesis supervisor Митропов Владимир Викторович, кандидат технических наук, Университет ИТМО, образовательный центр "Энергоэффективные инженерные системы", доцент (квалификационная категория "ординарный доцент")

Обучающийся/Student

Документ подписан
Жуков Андрей Дмитриевич
03.06.2024

(эл. подпись/ signature)

Жуков Андрей Дмитриевич
(Фамилия И.О./ name and surname)

Руководитель ВКР/ Thesis supervisor

Документ подписан
Митропов Владимир Викторович
30.05.2024

(эл. подпись/ signature)

Митропов Владимир Викторович
(Фамилия И.О./ name and surname)

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ / OBJECTIVES FOR A GRADUATION THESIS

Обучающийся / Student Жуков Андрей Дмитриевич
Факультет/институт/кластер/ Faculty/Institute/Cluster образовательный центр "Энергоэффективные инженерные системы"
Группа/Group W34601
Направление подготовки/ Subject area 14.03.01 Ядерная энергетика и теплофизика
Образовательная программа / Educational program Информационные технологии в энергетике 2020
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Тема ВКР/ Thesis topic Разработка лабораторного программного комплекса для студентов по определению параметров истечения идеального газа при дросселировании
Руководитель ВКР/ Thesis supervisor Митропов Владимир Викторович, кандидат технических наук, Университет ИТМО, образовательный центр "Энергоэффективные инженерные системы", доцент (квалификационная категория "ординарный доцент")

Характеристика темы ВКР / Description of thesis subject (topic)

Тема в области фундаментальных исследований / Subject of fundamental research: нет / not
Тема в области прикладных исследований / Subject of applied research: да / yes

Основные вопросы, подлежащие разработке / Key issues to be analyzed
Разработка лабораторного программного комплекса для студентов по определению параметров истечения идеального газа при дросселировании
Цель:
• Предоставить студентам возможность изучить процесс истечения идеального газа при дросселировании в интерактивной среде
• Предоставление студентам возможности самостоятельно провести измерения и расчеты, необходимые для определения параметров истечения газа.
• Закрепление теоретических знаний по теме "Дросселирование идеального газа".
Задачи:
• Провести анализ существующих методов определения параметров истечения идеального газа при дросселировании
• Подобрать и реализовать математические модели, описывающие процесс истечения идеального газа при дросселировании.
• Разработать программное обеспечение для лабораторного программного комплекса (виртуальной лабораторной работы).
• Разработать методические указания для студентов, содержащие описание работы с

виртуальной лабораторной установкой.
Структура работы:
1. Введение
2. Аналитический обзор
3. Разработка программного обеспечения
4. Методические указания
5. Заключение
6. Список литературы

Дата выдачи задания / Assignment issued on: 01.02.2024

Срок представления готовой ВКР / Deadline for final edition of the thesis 15.05.2024

СОГЛАСОВАНО / AGREED:

Руководитель ВКР/ Thesis supervisor	Документ подписан Митропов Владимир Викторович 12.03.2024 (эл. подпись)	Митропов Владимир Викторович
Задание принял к исполнению/ Objectives assumed BY	Документ подписан Жуков Андрей Дмитриевич 09.04.2024 (эл. подпись)	Жуков Андрей Дмитриевич
Руководитель ОП/ Head of educational program	Документ подписан Кораблев Владимир Антонович 13.05.2024 (эл. подпись)	Кораблев Владимир Антонович

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

АННОТАЦИЯ
ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ
SUMMARY OF A GRADUATION THESIS

Обучающийся / Student Жуков Андрей Дмитриевич
Факультет/институт/кластер/ Faculty/Institute/Cluster образовательный центр "Энергоэффективные инженерные системы"
Группа/Group W34601
Направление подготовки/ Subject area 14.03.01 Ядерная энергетика и теплофизика
Образовательная программа / Educational program Информационные технологии в энергетике 2020
Язык реализации ОП / Language of the educational program Русский
Квалификация/ Degree level Бакалавр
Тема ВКР/ Thesis topic Разработка лабораторного программного комплекса для студентов по определению параметров истечения идеального газа при дросселировании
Руководитель ВКР/ Thesis supervisor Митропов Владимир Викторович, кандидат технических наук, Университет ИТМО, образовательный центр "Энергоэффективные инженерные системы", доцент (квалификационная категория "ординарный доцент")

ХАРАКТЕРИСТИКА ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ
DESCRIPTION OF THE GRADUATION THESIS

Цель исследования / Research goal
1. Предоставить студентам возможность изучить процесс истечения идеального газа при дросселировании в интерактивной среде 2. Предоставление студентам возможности самостоятельно провести измерения и расчеты, необходимые для определения параметров истечения газа. 3. Закрепление теоретических знаний по теме "Дросселирование идеального газа".
Задачи, решаемые в ВКР / Research tasks
1. Проведение анализа существующих методов определения параметров истечения идеального газа при дросселировании 2. Подобрать и реализовать математические модели, описывающие процесс истечения идеального газа при дросселировании. 3. Разработать программное обеспечение для лабораторного программного комплекса (виртуальной лабораторной работы). 4. Разработать методические указания для студентов, содержащие описание работы с виртуальной лабораторной установкой.
Краткая характеристика полученных результатов / Short summary of results/findings
Разработанный лабораторный программный комплекс позволяет студентам: 1. Изучить процесс истечения идеального газа при дросселировании. 2. Самостоятельно, провести измерения и расчеты, необходимые для определения параметров истечения газа. 3. Сравнивать полученные результаты с теоретическими значениями 4. Закрепить теоретические знания по теме "Дросселирование идеального газа".

Обучающийся/Student

Документ подписан	
Жуков Андрей Дмитриевич	
03.06.2024	

(эл. подпись/ signature)

Жуков Андрей Дмитриевич

(Фамилия И.О./ name and surname)

Руководитель ВКР/
Thesis supervisor

Документ подписан	
Митропов Владимир Викторович	
30.05.2024	

(эл. подпись/ signature)

Митропов Владимир Викторович

(Фамилия И.О./ name and surname)

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....1
1.1. АКТУАЛЬНОСТЬ ТЕМЫ.....1
1.2 ЦЕЛЬ И ЗАДАЧИ ИССЛЕДОВАНИЯ.....2
1.3. ОБЪЕКТ И ПРЕДМЕТ ИССЛЕДОВАНИЯ.....2
ТЕОРЕТИЧЕСКИЙ ОБЗОР ФИЗИЧЕСКОЙ СОСТАВЛЯЮЩЕЙ.....3
2.1 ИСТЕЧЕНИЕ ГАЗОВ И ПАРОВ. ОСНОВНЫЕ ПОНЯТИЯ И МАТЕМАТИЧЕСКОЕ ОПИСАНИЕ.....3
2.2 АДИАБАТНОЕ ИСТЕЧЕНИЕ ИЗ СУЖИВАЮЩЕГОСЯ СОПЛА...11
2.3 ИСТЕЧЕНИЕ ЧЕРЕЗ КОМБИНИРОВАННОЕ СОПЛО. ПЕРЕХОД ЧЕРЕЗ СКОРОСТЬ ЗВУКА.....16
ТЕОРЕТИЧЕСКИЙ ОБЗОР ПРОГРАММНОЙ СОСТАВЛЯЮЩЕЙ.....20
3.1 HTML И CSS.....20
3.2 TYPESCRIPT.....21
3.3 NODE.JS.....21
3.4 NEST.JS.....23
3.5 ПОИСК В ШИРИНУ (BREADTH-FIRST SEARCH, BFS).....25
3.6 ЧИСТАЯ АРХИТЕКТУРА (РОБЕРТ МАРТИН).....28
ПРАКТИЧЕСКАЯ ЧАСТЬ.....32
4.1. РАЗРАБОТКА СТРУКТУРЫ И ИНТЕРФЕЙСА ЛАБОРАТОРНОГО ПРОГРАММНОГО КОМПЛЕКСА.....32
4.2 ОСОБЕННОСТИ РЕАЛИЗАЦИИ ГРАФИЧЕСКОГО МОДУЛЯ.....34
4.3 ОСОБЕННОСТИ РЕАЛИЗАЦИИ ФИЗИЧЕСКОГО МОДУЛЯ.....36
4.4 ОСОБЕННОСТИ РЕАЛИЗАЦИИ КОНФИГУРАЦИОННОГО МОДУЛЯ.....38
ДЕМОНСТРАЦИЯ ФУНКЦИОНАЛА ПРОГРАММНОГО КОМПЛЕКСА 43
5.1 ИНТЕРФЕЙС ИНИЦИАЛИЗАЦИИ ПРОГРАММНОГО КОМПЛЕКСА.....43
5.1 СБОРКА ЛАБОРАТОРНОЙ УСТАНОВКИ.....44
5.2 ЗАПУСК ЛАБОРАТОРНОЙ УСТАНОВКИ.....46
5.3 ОШИБКИ ВОЗМОЖНЫЕ ПРИ ЗАПУСКЕ.....48
ЗАКЛЮЧЕНИЕ.....50

ВВЕДЕНИЕ

1.1. АКТУАЛЬНОСТЬ ТЕМЫ

Современное образование в области технических наук находится в постоянном процессе поиска новых эффективных способов подачи материала, повышения его доступности и понятности для студентов в рамках стандартизации образовательного процесса. Ведь для закрепления и профессионального понимания сложных научных процессов, важно не просто знать теорию, но и уметь применять её на практике, видеть живую реализацию принципов и правил, лежащих в её основе. Только полноценное погружение в суть процесса позволяет надежно "усвоить" его, перейдя от абстрактного понимания к конкретным умениям и навыкам.

Однако, многие из этих процессов так или иначе связаны с реальными физическими или техническими явлениями и могут быть достаточно сложны для непосредственного наблюдения или экспериментирования в аудиторных условиях. Здесь на помощь приходит компьютерное моделирование - мощный инструмент, позволяющий создавать достоверные виртуальные "копии" реальных процессов, воспроизводя их со всеми особенностями и деталями.

В данной квалификационной работе рассматривается один из таких процессов - истечение идеального газа при дросселировании, который является значимой частью курса по термодинамике. Изучение данного процесса требует точного понимания и умения рассчитать различные параметры, что может стать сложной задачей для студентов без соответствующего практического опыта.

1.2 ЦЕЛЬ И ЗАДАЧИ ИССЛЕДОВАНИЯ

Целью проекта является разработка программного комплекса, который сможет показать процессы истечения идеального газа при дросселировании, а также дать студентам возможность практически применять свои знания для определения и расчета соответствующих параметров. Будет разработан образовательный инструмент, который будет одинаково полезен как для студентов, так и для преподавателей, помогая первым наглядно увидеть и понять процессы, которые они изучают теоретически, а вторым - контролировать прогресс и качество обучения в динамике.

1.3. ОБЪЕКТ И ПРЕДМЕТ ИССЛЕДОВАНИЯ

Объектом исследования в работе является процесс дросселирования идеального газа. Это явление характеризуется изменением параметров газа при прохождении через узкое отверстие или диафрагму, что в свою очередь влияет на процесс его истечения, что будет подробно рассмотрено в рамках проведения исследования.

Предметом исследования является разработка программного комплекса, который будет эффективно моделировать данный процесс, позволяя определить ключевые параметры истечения газа при его дросселировании. Это подразумевает анализ теоретической базы, выбор оптимальных методов и алгоритмов для моделирования и расчетов, а также разработку удобного для пользователя интерфейса.

ТЕОРЕТИЧЕСКИЙ ОБЗОР ФИЗИЧЕСКОЙ СОСТАВЛЯЮЩЕЙ

2.1 ИСТЕЧЕНИЕ ГАЗОВ И ПАРОВ. ОСНОВНЫЕ ПОНЯТИЯ И МАТЕМАТИЧЕСКОЕ ОПИСАНИЕ

Рассмотрим истечение газа (или пара) из сосуда бесконечно большой емкости через канал переменного сечения. При этом будем полагать, что начальные параметры, например температура и давление газа (или пара), в сосуде остаются в процессе истечения неизменными; а начальная его скорость близка к нулю.

Истечение газа через канал происходит в результате перепада давлений вдоль оси канала: давление на входе в канал отличается от давления среды, в которую происходит истечение (в устье канала). Естественно, что при равенстве давлений внутри канала истечение невозможно.

В соответствии с существующей технической терминологией каналы, в которых движется рабочее тело (газ или пар), носят названия сопел и диффузоров. Сопло — это канал, в котором потенциальная энергия потока превращается в кинетическую, предназначенный для получения высоких скоростей движущегося потока. [1] Диффузор — канал, обеспечивающий торможение потока газа (или пара), сжатие рабочего тела, играющий роль своеобразного компрессора. [2] В первом случае (сопло) давление газа на входе в канал значительно больше давления в устье канала. Во втором случае (диффузор) — картина обратная.

Основные закономерности процесса истечения. Уравнение (1) первого закона термодинамики:

$$dq = du + dl \tag{1}$$

имеет общий характер и справедливо для любых систем — неподвижных и движущихся. Однако в случае движущихся систем, т. е. если рабочее тело

перемещается в пространстве со значительной скоростью (десятки и сотни метров в секунду), необходимо учитывать некоторые дополнительные обстоятельства.
Работа изменения объема газа расходуется на перемещение газа в канале (преодоление сопротивления внешней среды) и на приращение кинетической энергии (скорости) потока газа[3] (2):

$$dl = dl' + d\left(\frac{w^2}{2}\right) \tag{2}$$

Здесь dl' — часть работы dl, затрачиваемой на перемещение газа в канале — работа проталкивания; $d\left(\frac{w^2}{2}\right)$ — изменение кинетической энергии потока. Производя замену в уравнении (1) в соответствии с (2), получаем:

$$dq = du + dl' + d\left(\frac{w^2}{2}\right) \tag{3}$$

Выражение (3) представляет собой уравнение первого закона термодинамики для потока газа в одномерном приближении.

Действие соседних слоев на выделенный объем заменим внешними силами: pf — действует на левую поверхность слоя с площадью f и $(p+dp)(f+df)$ — на правую поверхность слоя с площадью $f+df$. Допустим, что общее направление движения потока совпадает с осью канала направо. Тогда для перемещения выделенной массы потока рабочее тело в указанном направлении потребуется за единицу времени совершить работу, равную алгебраической сумме работы, которую производят внешние силы над потоком (знак минус), и работы, которую производит поток (эта работа считается положительной) над внешними силами:

$$dL' = (p+dp)(f+df)(w+dw) - pfw, \tag{4}$$

где p — давление; w — скорость; L' — работа, отнесенная к массе газа.

После сокращения и отбрасывания малых величин второго и высшего порядка получаем:

$$dL' = pfw + pwdf + wfdp = pd\left(vw\right) + wfdp. \tag{5}$$

Для установившегося потока количество рабочего тела, проходящее через любое сечение канала площадью f в единицу времени, равно

$$M = \frac{fw}{v}, \tag{6}$$

где M — секундный массовый расход рабочего тела; v — удельный объем газа или пара в данном сечении канала. Учитывая, что расход M — величина постоянная, уравнение (5) можно привести к виду:

$$dL' = pdv = vdp = d(pv), \tag{7}$$

так как $d(fw) = du$ и $wf = v$. С учетом (7) уравнение (2) можно записать в виде

$$pdv = d(pv) + d\left(\frac{w^2}{2}\right). \tag{8}$$

Располагаемая работа

$$dl - dl' = d\left(\frac{w^2}{2}\right). \tag{9}$$

Превратить в другие виды работы можно лишь ту часть работы расширения, которая расходуется на увеличение кинетической энергии потока. Величину $d(\frac{w^2}{2}) = vdp = dl - dl'$ называют располагаемой работой и обозначают dq, т. е.

$$dl - dl' = dq. \tag{10}$$

Интегрируя (8) с учетом (10), имеем следующее соотношение:

$$l = l' + l_0, \tag{11}$$

$$l_0 = \left(\frac{w_2^2 - w_1^2}{2}\right) / l_1, \tag{12}$$

$$\int_{p_1}^{p_2} vdp = \int_{p_1}^{p_2} \frac{Q}{l}. \tag{13}$$

Рассмотрим конкретные аналитические зависимости для определения располагаемой работы наиболее распространенных термодинамических процессов идеального газа.

Если расширение происходит по политропе с уравнением $pv^n = \mathrm{const}$, то работа расширения, как известно, равна

$l = \frac{1}{n-1}(p_1 v_1 - p_2 v_2).$

Тогда, поскольку из уравнения (7) имеем $l' = p_2 v_2 - p_1 v_1$, получим, следуя (12):

$$l_0 = l - l' = \frac{n}{n-1}(p_1 v_1 - p_2 v_2) = nl. \tag{14}$$

В зависимости от значения показателя политропы n располагаемая работа может быть больше, равна или меньше работы расширения.

Так, в изотермическом процессе $n = 1$ и $l_0 = l$, т. е. работа расширения равна располагаемой работе. В процессе адиабатного расширения $n = k$ располагаемая работа всегда больше работы расширения и согласно формуле (13) равна $l_0 = (p_1 - p_2)v$.

Подставив в (3) выражение для работы проталкивания (7), получим

$$dq = du + d(pv) + d\left(\frac{w^2}{2}\right) = d(u + pv) + d\left(\frac{w^2}{2}\right). \tag{15}$$

Поскольку $u + pv = h$, имеем

$$dq = dh + d\left(\frac{w^2}{2}\right), \tag{16}$$

или

$$dq = dh + dl_0. \tag{17}$$

Из уравнения (10.12) следует важное заключение. Поскольку $dl_0 = dq - dh$, для адиабатного течения газа $dq = 0$ имеем

$$-dh = dl_0 = d\left(\frac{w^2}{2}\right), \tag{18}$$

$$l_0 = h_1 - h_2. \tag{19}$$

В адиабатном процессе приращение энтальпии равно располагаемой работе потока. Из уравнения (18) также видно, что уменьшение энтальпии потока сопровождается его ускорением, и, наоборот, торможение потока соответствует увеличению его энтальпии. Это позволяет в этом случае классифицировать энтальпию как потенциальную энергию потока.

.

При течении газа (жидкости) в канале в условиях установившегося, стационарного режима истечения для любого сечения канала справедливы соотношения:

$$M = \frac{fw}{v} = \mathrm{const}, \tag{20}$$

называемое уравнением сплошности, или уравнением неразрывности потока. Прологарифмируем его $\ln f + \ln w - \ln v = \mathrm{const}$, далее продифференцировав это соотношение, получим

$$\frac{df}{f} = \frac{dv}{v} - \frac{dw}{w}. \tag{21}$$

Это и есть уравнение неразрывности в дифференциальной форме.

Обратим внимание на особенности уравнения (21). При расширении рабочего тела $dv/v > 0$ и $dw/w > 0$ (увеличение скорости движения потока в сопле). Возможен случай, когда $dv/v > dw/w$ и $dv/v < dw/w$. В первом случае $df/f > 0$ (канал расширяется), во втором $df/f < 0$ (сечение канала суживается по ходу течения потока). При сжатии рабочего тела (процесс в диффузоре), $dv/v < 0$ и $dw/w < 0$ (торможение потока). В зависимости от

соотношения этих величин в уравнении (21) канал диффузора может быть как расширяющимся, так и суживающимся.

Исследуем уравнение (21) для случая обратимого адиабатного (s = const) истечения идеального газа. Найдём значение dv/v и dw/w. При отсутствии теплообмена для идеального газа справедливо уравнение адиабаты:

$$pv^k = \text{const}, \tag{22}$$

откуда, дифференцируя, имеем

$$pkv^{k-1}dv + v^k dp = 0. \tag{23}$$

Поделив на v^k, запишем $dp/v = -\left(\frac{k}{v}\right) dp/p$.

Имеем $dv/v = -\left(\frac{v}{w^2}\right) dp$. Подставив значения dv/v и dw/w в уравнение (21), получим:

$$\frac{df}{f} = \frac{1}{k}\frac{dp}{p} - \frac{kpv - w^2}{kw^2}\frac{dp}{p}, \tag{24}$$

Как известно, местная скорость звука в газе или паре, имеющем состояние параметра p и v,

$$a = \sqrt{kpv}. \tag{25}$$

С учетом (25) уравнение (24) преобразуем к виду

$$\frac{df}{f} = \left(\frac{a^2 - w^2}{kw^2}\right)\frac{dp}{p}, \tag{26}$$

или, что то же самое,

$$\frac{df}{f} = \frac{1}{k}\left(\frac{1}{\mathrm{Ma}^2} - 1\right)\frac{dp}{p}. \tag{27}$$

Отношение скорости истечения w к местной скорости звука a называется числом Маха (Ма). Если < 1, то скорость истечения называют дозвуковой; если > 1 — сверхзвуковой.

2.2 АДИАБАТНОЕ ИСТЕЧЕНИЕ ИЗ СУЖИВАЮЩЕГОСЯ СОПЛА

Пусть в резервуаре, размеры которого достаточно большие, находится газ, вытекающий через суживающееся сопло. Обозначим параметры газа в резервуаре через p_1, v_1, T_1. Значения этих параметров из-за размеров резервуара со временем не изменяются. Давление среды, в которую вытекает газ, обозначим через p_2. Начальную скорость газа в резервуаре примем равной нулю $(v_1 = 0)$. Определим скорость истечения газа из сопла и его расход.

Скорость, с которой рабочее тело вытекает из сопла, составляет практически несколько сотен метров в секунду. При такой скорости истечение газа или пара происходит столь быстро, что рабочее тело не успевает ни получить от окружающей среды, ни отдать ей сколько-нибудь значительное количество теплоты. [4] Поэтому правомерно считать такой процесс истечения адиабатным.

Для обратимого адиабатного истечения располагаемая работа вычисляется по уравнению:

$$l_0 = h_1 - h_2 = kl, \tag{27}$$

или, поскольку $l = \dfrac{1}{k-1}(p_1 v_1 - p_2 v_2)$, имеем:

$$l_0 = \frac{k}{k-1} p_1 v_1 \left[1 - \left(\frac{p_2}{p_1} \right)^{\frac{k-1}{k}} \right]. \tag{28}$$

Получаем для скорости истечения газа из сопла:

$$w_2 = \sqrt{2\frac{k}{k-1}p_1v_1\left[1-\left(\frac{p_2}{p_1}\right)^{\frac{k-1}{k}}\right]}. \tag{29}$$

Массовый расход газа через сопло может быть найден из уравнения сплошности:

$$M = f_2w_2/v_2, \tag{30}$$

где f_2 — площадь выходного сечения сопла, v_2 — удельный объем газа в выходном сечении.

Поскольку для адиабатного процесса $p_1v_1^k = p_2v_2^k$, то

$$1/v_2 = \left(\frac{p_2}{p_1}\right)^{1/k}\left(\frac{1}{v_1}\right) \tag{31}$$

и

$$M = (f_2w_2/v_1)\left(\frac{p_2}{p_1}\right)^{1/k}. \tag{32}$$

Подставляя сюда значение w_2 из уравнения, имеем:

$$M = f_2\sqrt{2\frac{k}{k-1}p_1v_1\left[\left(\frac{p_2}{p_1}\right)^{2/k}-\left(\frac{p_2}{p_1}\right)^{\frac{k+1}{k}}\right]}. \tag{33}$$

Следует, что для газа при данных начальных параметрах p_1, v_1 и сечении сопла на выходе f_2 расход и скорость истечения зависят только от давления среды, в которую происходит истечение, точнее, от отношения давлений p_2/p_1. Действительно, при $p_2/p_1 = 1$, т.е. $p_2 = p_1$, $M = 0$. В то же время при истечении в вакуум $(p_2 = 0)$ получим, что $M = 0$. Из него видно, что при $f_2 = \text{const}$ секундный расход зависит от скорости w_2 и удельного объема газа v_2. С уменьшением давления p_2 скорости w_2 и удельный объем увеличиваются. Однако увеличение скорости w_2 с уменьшением давления растет быстрее, чем объем v_2, поэтому расход M также растет. Достигнув некоторого максимального значения, расход газа начинает уменьшаться. Это происходит потому, что при дальнейшем уменьшении p_2 скорость истечения w_2 растет медленнее, чем удельный объем v_2. Так, при $p_2 = 0$ скорость будет иметь конечное значение, хотя $v_2 \to \infty$, т.е. $M \to 0$.

Давление p_2 на срезе сопла, при котором расход газа достигает максимума, называется критическим. Отношение p/p_1 обозначают через β. Скорость истечения газа из суживающегося сопла при максимальном расходе называется критической и обозначается w_2.

$$2/(k+1) = \left(\frac{p_2}{p_1}\right)^{(k-1)/k} = \beta = \left[\frac{2}{k+1}\right]^{k/(k-1)}. \tag{34}$$

β целиком определяется значением показателя адиабаты k, т.е. физическими свойствами вытекающего газа. Так, при $k = 1.4$, $\beta = 0.538$; при $k = 1.3$, $\beta = 0.55$. Можно в первом приближении считать $\beta = 0.5$.

Получим формулы для определения w_2 и M :

$$w_2 = \sqrt{2\left[\frac{k}{k-1}\right] p_1 v_1}, \tag{35}$$

$$M = f_2 \sqrt{\frac{2k}{k-1}\frac{p_1}{v_1}\left(\left[\frac{2}{k+1}\right]^{2/(k-1)} - \left[\frac{2}{k+1}\right]^{(k+1)/(k-1)}\right)}. \tag{36}$$

Для удельного объема v_1 из уравнения адиабаты в случае, когда в выходном сечении сопла устанавливаются параметры p_2 v_2, имеем:

$$v_1 = v_2 \left(\frac{2p_1}{p_1}\right)^{1/k} = v_2 \beta^{1/k}. \tag{37}$$

Заменяя β , получаем:

$$v_1 = 2v \left(\frac{2}{k+1}\right)^{1/(k-1)}, \tag{38}$$

и для давления p_1:

$$p_1 = 2p \left(\frac{k+1}{2}\right)^{(k-1)/k}. \tag{39}$$

Критическая скорость истечения w_2 равна местной скорости звука в выходном сечении сопла. Другими словами, получить с помощью сопла более высокую (сверхзвуковую) скорость потока невозможно.

Любое слабое возмущение, например уменьшение давления, распространяется по струе газа со скоростью, равной скорости звука. При

малых скоростях истечения уменьшение внешнего давления передается по струе газа внутрь сопла и приводит к перераспределению давления в сопле. В этом случае давление в выходном сечении сопла уменьшается до значения внешнего давления, а скорость и расход, соответственно, возрастают. Но как только скорость газа на выходе из сопла достигает скорости звука, никакое уменьшение давления окружающей среды по струе газа передать невозможно. При уменьшении давления за соплом ниже p_2 скорость и давление в выходном сечении сопла остаются теми же (w_2 и p_2), независимо от величины внешнего давления p_2. Перепад давления от p_2 до p_1 остается неиспользованным для увеличения кинетической энергии струи вытекающего газа и почти полностью тратится на образование вихрей при истечении.

Приступая к расчету истечения идеального газа из сужающегося сопла при заданных значениях p_1 и p_2, следует прежде всего сравнить отношение p_2/p_1 с величиной β.

Для паров качественная картина процесса истечения не изменяется. Однако расчетные формулы, в которые входит величина показателя адиабаты k, будут приближенными, поскольку его значение для пара в процессе истечения не сохраняется постоянным. Поэтому применяют соотношения:

$$l_0 = h_1 - h_2 : \tag{40}$$

$$w_2 = \sqrt{2(h_1 - h_2)}, \tag{41}$$

$$M = \left(\frac{f_2}{v_2 k p_2}\right)\sqrt{2(h_1 - h_2)}, \tag{42}$$

если $\frac{p_2}{p_1} \leq B$. Характеристики истечения в случае $\frac{p_2}{p_1} \leq B$ определяются из уравнений:

$$w_2 = \sqrt{2(h_1 - h_2)}, \tag{43}$$

$$M_{max} = \left(\frac{f_2}{v_2 k p_2}\right)\sqrt{2(h_1 - h_2)}. \tag{44}$$

Такие параметры пара на выходе из сопла, как удельный объем, температура, энтальпия и другие, могут быть найдены для обратимого адиабатного истечения из диаграммы $i - s$. Для этого из точки I, соответствующей начальному состоянию пара перед истечением, нужно провести адиабату расширения $1 - 2$ до пересечения с изобарой p_2, если $\frac{p_2}{p_1} > B$, или с изобарой p_2, если $\frac{p_2}{p_1} \leq B$.

В термодинамических соотношениях, описывающих процесс истечения, фигурирует только одна геометрическая характеристика канала, а именно величина выходного сечения канала f_2.

2.3 ИСТЕЧЕНИЕ ЧЕРЕЗ КОМБИНИРОВАННОЕ СОПЛО. ПЕРЕХОД ЧЕРЕЗ СКОРОСТЬ ЗВУКА

В случае, когда для суживающегося сопла значение скорости не превышает местной скорости звука, то есть

$$\frac{p_2}{p_1} \leq \beta, \tag{45}$$

где β — критическое отношение давления, соответствующее скорости звука в данной точке $w = a$. Давление среды p_2, куда происходит истечение, может быть значительно ниже p_2.

Из анализа, проведенного ранее, становится очевидным, что полезно использовать весь перепад давлений от p_1 до p_2. Для этого канал должен иметь после суживающегося участка расширяющийся участок.

Давление истечения p_2 достигает местной скорости звука. За этим сечением канал должен быть выполнен расширяющимся. Поток в этом случае перейдет через скорость звука (Ма > 1) и будет продолжать ускоряться в расширяющейся части сопла. Для ускорения потока будет использован весь перепад давлений от p_1 до p_2, а не часть этого перепада, реализуемая в суживающемся дозвуковом сопле.

Комбинированное сопло, состоящее из суживающейся и расширяющейся частей, впервые было применено шведским инженером К. Г. П. де Лавалем и получило название сопла Лаваля. Сопло Лаваля используется в случаях, когда

$$\frac{p_2}{p_1} \leq \beta. \tag{46}$$

В наименьшем сечении (горле) давление потока равно $p_2 = \beta p_1$, а скорость движения достигает $w_2 = a$. Площадь минимального сечения определяется из соотношения

$$f_{\min} = \left(\frac{M_{\max} \nu_2}{w_2} \right). \tag{47}$$

Скорость w_2 определяют из вышеуказанных формул.

Особо отметим, что сопло Лаваля позволяет получить только сверхзвуковую скорость истечения, но максимальная пропускная способность сопла не

изменяется и определяется по величине ν_2. Поскольку расход газа в горловине сопла и на его срезе один и тот же, можно записать

$$\left(\frac{f_{\min} w_2}{\nu_2}\right) = \left(\frac{f_{\max} w_2}{\nu_2}\right), \tag{48}$$

откуда легко найти величину выходного сечения сопла Лаваля

$$f_{\max} = f_{\min}\left(\frac{w_2 \nu_2}{w_2 \nu_2}\right) = f_{\min}\left(\frac{M_{\max} \nu_2}{w_2 \nu_2}\right). \tag{49}$$

Величина удельного объема ν_2 газа определяется из уравнения адиабаты

$$\nu_2 = \left(\frac{p_1}{p_2}\right)^{1/k} \nu_1, \tag{50}$$

где k — показатель адиабаты.

Непременным условием получения скорости истечения, превышающей скорость звука, является отсутствие отрыва струи от стенок сопла. Для этого необходимо, чтобы угол конусности α расширяющейся части сопла был не более 10—12°. Длина h_2 расширяющейся части сопла Лаваля определяется по формуле

$$h_2 = \frac{(d_{\max} - d_{\min})}{(2\,\mathrm{tg}(\alpha/2))}, \tag{51}$$

где $d_{\max}$ и $d_{\min}$ — диаметры в выходном сечении и в горловине сопла. Длина суживающейся части сопла h_1 выбирается минимальной.

Сопло Лаваля обеспечивает преобразование всей работы истечения в кинетическую энергию струи. Поэтому располагаемая работа l_0 будет равна всей площади, заключенной между адиабатой и осью ординат. [5]

ТЕОРЕТИЧЕСКИЙ ОБЗОР ПРОГРАММНОЙ СОСТАВЛЯЮЩЕЙ

3.1 HTML И CSS

HTML и CSS[6] - две основные технологии, используемые в веб-разработке. HTML - это язык разметки, а CSS - язык таблиц стилей.

HTML использует теги для определения структуры веб-страницы. Эти теги сообщают браузеру, как отображать содержимое, такое как заголовки, абзацы, списки, изображения и ссылки. Теги HTML заключены в угловые скобки, и они идут парами, с открывающим тегом и закрывающим тегом.

CSS, с другой стороны, используется для применения стилей к содержимому HTML. Это включает различные свойства, такие как цвет, размер, шрифт, интервалы и компоновку. CSS позволяет более гибко управлять дизайном и компоновкой веб-страницы, позволяя разработчикам создавать визуально привлекательные и удобные для пользователя веб-сайты.

CSS может быть применен к HTML-элементам несколькими способами. Его можно добавить встроено в HTML-документ, во внутренний стиль в секции head HTML-документа, или во внешнюю таблицу стилей, связанную с HTML-документом. Внешние таблицы стилей являются наиболее часто используемым методом, поскольку они обеспечивают согласованность на нескольких веб-страницах и более простое обслуживание кодовой базы.

Разделение содержимого и представления - ключевой принцип веб-разработки, достигаемый через использование HTML для содержимого и CSS для стилизации. Это разделение позволяет более гибко и легко работать с дизайном, поскольку изменения в дизайне могут быть внесены в CSS без влияния на содержимое в HTML.

Более того, CSS также может быть использован для создания адаптивного дизайна, который адаптируется к разным размерам экрана и устройствам.

3.2 TYPESCRIPT

TypeScript — это типизированное надмножество JavaScript, которое добавляет к JavaScript статическую типизацию, классы, интерфейсы и другие функции, облегчающие разработку больших приложений. [6] TypeScript позволяет разработчикам писать более надежный и понятный код, а также обеспечивает более строгую проверку типов во время компиляции.

Основные концепции TypeScript:

1. Статическая типизация
2. Классы и интерфейсы
3. Модули
4. Аннотации типов
5. Интерфейсы
6. Generics

Typescript компилируется в javascript файл, что позволяет легко запускать приложение на любой машине.

Преимущества использования TypeScript:

1. Улучшение качества кода
2. Упрощение разработки
3. Повышение производительности

3.3 NODE.JS

Node.js — это среда выполнения JavaScript, которая позволяет выполнять JavaScript код не в браузере (на сервере).

Основные особенности Node.js:

1. Неблокирующий ввод-вывод. Node.js использует неблокирующий ввод-вывод, что позволяет обрабатывать множество запросов одновременно. Это делает Node.js идеальным выбором для создания масштабируемых серверных приложений.
2. Обработка событий. Node.js основан на модели обработки событий, что позволяет эффективно обрабатывать запросы. Это упрощает разработку серверных приложений и делает их более гибкими.
3. Модульная система

Преимущества использования Node.js:

1. Масштабируемость. Неблокирующий ввод-вывод и обработка событий делают Node.js масштабируемым решением для создания серверных приложений.
2. Скорость разработки. Модульная система и большое сообщество разработчиков делают Node.js быстрым решением для разработки серверных приложений.
3. Низкие затраты на инфраструктуру.

Node.js является мощным инструментом для создания серверных приложений. Он предоставляет разработчикам возможность создавать масштабируемые, быстрые и эффективные приложения.

Примеры использования Node.js:

1. Веб-серверы. Node.js используется для создания веб-серверов, таких как Express.js и Next.js.
2. API-серверы. Node.js используется для создания API-серверов, которые предоставляют данные и функции другим приложениям.
3. Серверы приложений. Node.js используется для создания серверов приложений, которые обрабатывают запросы и возвращают данные.
4. Приложения реального времени. Node.js используется для создания чат-приложений, игровых серверов и других приложений реального времени.

3.4 NEST.JS

Nest.js[9] построен на основе фреймворка Express.js, популярного веб-фреймворка для приложений Node.js. Используя мощь и гибкость Express.js, Nest.js предоставляет структурированный и субъективный подход к созданию приложений, который отлично подходит как для небольших проектов, так и для крупных корпоративных приложений.

Важная особенность Nest.js - то, что он поддерживает модули. Модули в Nest.js - это способ организации и инкапсуляции связанного кода в приложении. Разбивая ваше приложение на отдельные модули, можно создать более структурированный и поддерживаемый код. Модули в Nest.js также могут легко использоваться и повторно использоваться в разных проектах, что облегчает создание и поддержку библиотеки повторно используемых компонентов.

Nest.js также обеспечивает поддержку промежуточного программного обеспечения, функции которого могут выполняться перед или после основной логики обработчика маршрута. Промежуточное программное обеспечение в Nest.js может использоваться для выполнения задач, таких как ведение журнала, аутентификация, обработка ошибок и многое другое. Позволяя разработчикам определять и применять промежуточное программное обеспечение в модульном и повторно используемом виде, Nest.js упрощает добавление общей функциональности в приложение без дублирования кода.

Еще одной ключевой функцией Nest.js является встроенная поддержка TypeScript, добавляющего статическую типизацию и другие функции к языку. Используя TypeScript с Nest.js, можно обнаружить ошибки на этапе компиляции, воспользоваться расширенными функциями языка и написать более поддерживаемый и читаемый код. TypeScript также интегрируется плавно с Nest.js, обеспечивая гладкий процесс разработки для разработчиков, предпочитающих статическую типизацию.

Nest.js включает мощный интерфейс командной строки (CLI), который облегчает создание новых проектов, модулей, контроллеров, услуг и многого другого. С помощью CLI можно также создавать и запускать приложения Nest.js, облегчая разработку и тестирование кода локально перед развертыванием его в производственной среде. CLI разработан для стандартизации общих задач разработки и помощи разработчикам быть более продуктивными.

Nest.js также поставляется с системой тестирования, которая позволяет легко писать и запускать автоматизированные тесты для приложений. Система

тестирования в Nest.js поддерживает: юнит-тесты, интеграционные тесты и тесты конечных точек.

3.5 ПОИСК В ШИРИНУ (BREADTH-FIRST SEARCH, BFS)

Поиск в ширину — это алгоритм на графах, который используется для обхода вершин графа и поиска пути между двумя заданными вершинами. [10]

Идея алгоритма поиска в ширину заключается в том, что мы заходим в вершину и исследуем все смежные с ней вершины, потом мы переходим в следующую вершину и делаем тоже самое что было описано прежде.

Алгоритм работает следующим образом:

1. Создаётся очередь, в которую помещается начальная вершина.
2. Пока очередь не пуста, выполняются следующие шаги:
    a. Из очереди извлекается текущая вершина.
    b. Проверяется, является ли текущая вершина конечной. Если да, то алгоритм завершается успешно.
    c. Для текущей вершины находятся все смежные вершины, которые ещё не были исследованы. Эти вершины помещаются в очередь.
3. Если конечная вершина не была найдена, то алгоритм завершается неудачно.

Поиск в ширину:

1. Является корректным, то есть всегда находит путь между двумя заданными вершинами, если такой путь существует.
2. Является полным, то есть исследует все вершины графа.
3. Имеет сложность O(n + m), где n — количество вершин графа, а m — количество рёбер графа.

Применение поиска в ширину:

1. Поиск кратчайшего пути между двумя вершинами в невзвешенном графе.
2. Проверка связности графа.
3. Нахождение компонент связности графа.
4. Поиск всех вершин, достижимых из заданной вершины.

```
src > app > steps > process.ts > ...
class Node<T> {
    data: T;
    adjNodes: Node<T>[];
    comparator: (a: T, b: T) => number;

    constructor(data: T, comparator: (a: T, b: T) => number) {
        this.data = data;
        this.adjNodes = new Array<Node<T>>();
        this.comparator = comparator;
    }
}

class Graph<T> {
    nodes: Map<T, Node<T>> = new Map<T, Node<T>>();
    comparator: (a: T, b: T) => number;
    root: Node<T>;

    constructor(comparator: (a: T, b: T) => number, data: T) {
        this.comparator = comparator;
        this.root = new Node<T>(data, comparator);
    }
}

export function bfs<T> (graph: Graph<T>, startNode: Node<T>){
    if (!startNode){
        return;
    }
    const visited: Node<T>[] = [];
    const queue: Node<T>[] = [];

    queue.push(startNode);

    while (queue.length !== 0){
        let current: Node<T>|undefined = queue.shift();
        if (!current){
            continue;
        }

        console.log(current.data);

        visited.push(current);
        current.adjNodes.forEach(node => {
            // if the node has not been visited
            if (visited.indexOf(node) === -1){
                visited.push(node);
                queue.push(node);
            }
        })
    }
}
```

Рисунок 1 - Реализация алгоритма поиск в ширину

3.6 ЧИСТАЯ АРХИТЕКТУРА (РОБЕРТ МАРТИН)

Чистая архитектура, также известная как «The Clean Architecture», была впервые предложена Робертом Мартином, опытным программистом и автором множества книг по разработке ПО. Основу ее идеологии составляет глубокое сосредоточение на бизнес-логике, а не на деталях и технических аспектах конкретных фреймворков или библиотек.

Основополагающим принципом чистой архитектуры является то, что архитектура приложения должна быть независима от механизмов ввода/вывода, баз данных, веб-серверов и любой другой внешней сущности. Это обеспечивает высокую гибкость и упрощает процесс адаптации системы к изменениям во внешнем окружении.

Модульность и разделение ответственности являются другими ключевыми принципами чистой архитектуры. Мартин акцентирует внимание на том, что бизнес-логика должна быть изолирована в собственных модулях, которые не зависят от внешних источников данных или пользовательского интерфейса.

Чистая архитектура также сильно подчеркивает важность тестирования. Если архитектура системы хорошо спроектирована, тестирование становится простым делом, поскольку каждый модуль или компонент могут быть протестированы независимо и изолировано от других.

Наконец, в основе этого подхода лежит стремление сделать архитектуру простой и понятной. Это значит, что архитектура должна быть разработана таким образом, чтобы разработчики могли легко понять ее структуру, принципы работы и как каждый компонент взаимодействует с другими. Это важное качество облегчает поддержку системы на протяжении всего ее жизненного цикла.

Чистая архитектура, предложенная Робертом Мартином, структурирована вокруг четырех основных компонентов: «Энтити», «Используемые случаи», «Интерфейсы и Адаптеры» и «Слои Фреймворков и Драйверов».

1. Энтити: Энтити представляют элементы бизнес-логики, или предметную область приложения. Они содержат критически важные бизнес-правила, которые являются связующим звеном всей системы. Энтити не зависят от конкретных деталей внешних слоев или функциональностей. Например, в системе управления финансами, «счет» или «транзакция» могут быть энтити.

2. Используемые случаи: Используемые случаи или сценарии отвечают за реализацию специфичных бизнес-логик и показывают, как энтити взаимодействуют друг с другом для достижения бизнес-целей. Они выступают в качестве посредников между внешними воздействиями (например, запросы от пользователей) и внутренними энтити.

3. Интерфейсы и Адаптеры: Интерфейсы и Адаптеры служат связующим звеном между верхними уровнями (энтити и используемые случаи) и фреймворками и драйверами нижнего уровня. Они преобразуют данные в формат, который может быть использован энтити и сценариями использования. Например, они могут конвертировать данные из базы данных в формат, подходящий энтити.

4. Слои Фреймворков и Драйверов: Это наименее стабильный слой в иерархии чистой архитектуры. Он содержит внешние фреймворки и библиотеки, такие как базы данных, веб-серверы, библиотеки для работы с пользовательским интерфейсом и т.д.

Внутренние слои не содержат никакой информации о внешних слоях. Например, энтити не должны знать ничего о базе данных или пользовательском интерфейсе. Это обеспечивает строгий контроль над контрактами между слоями, обеспечивает устойчивость системы к изменениям и упрощает тестирование системы.

Преимущества и недостатки чистой архитектуры

Преимущества:

1. Независимость от UI: Одним из значительных преимуществ чистой архитектуры является то, что бизнес-логика системы остается независимой от пользовательского интерфейса. Это обуславливает возможность применения различных пользовательских интерфейсов без изменения основной логики программы.

2. Независимость от базы данных: Аналогично, система, построенная с применением чистой архитектуры, не связана с конкретной базой данных. Это предоставляет свободу выбора или изменения базы данных без затрагивания основной логики приложения.

3. Тестируемость: Так как компоненты в чистой архитектуре имеют ясно определенные границы и минимальные зависимости, это облегчает написание автоматических тестов без необходимости создания сложных заглушек.

4. Гибкость: Чистая архитектура обеспечивает гибкость системы благодаря своему модульному и слоевому подходу. Это означает, что изменения в одной части системы могут быть осуществлены относительно легко и без влияния на другие части.

Недостатки:

1. Сложность: Применение чистой архитектуры может быть сложно для маленьких проектов или для команд, незнакомых с этим подходом. Это может привести к избыточности и усложнению кода, если не обрабатывается должным образом.

2. Время на разработку

3. Необходимость в обучении

Несмотря на эти недостатки, чистая архитектура считается одним из самых прочных и устойчивых подходов к архитектуре программного обеспечения, и ее преимущества зачастую перевешивают возможные трудности.

Практическое использование чистой архитектуры

Чистая архитектура может быть применена к любому проекту разработки программного обеспечения, будь то веб-приложения, десктопные приложения, микросервисы или сложные распределенные системы. Ниже приведены некоторые из ключевых аспектов ее практического использования.

1. Идентификация и проектирование "Энтити": Первый шаг в применении чистой архитектуры - определение ключевых энтити вашего приложения. Эти энтити представляют ваши основные бизнес-объекты и содержат наиболее критические бизнес-правила.

2. Определение "Используемых случаев": Затем идентифицируются различные бизнес-процессы или "используемые случаи", которые описывают, как энтити взаимодействуют друг с другом, чтобы достичь бизнес-целей.

3. Проектирование "Интерфейсов и Адаптеров": Этот слой отвечает за преобразование данных из формата, используемого энтити и используемыми случаями, в формат, подходящий для внешних слоев, таких как базы данных или веб-серверы.

4. Разработка "Слоев Фреймворков и Драйверов": Наконец, проекты выбирают и интегрируют подходящие технологии, такие как фреймворки веб-приложений, библиотеки работы с базами данных, системы управления очередями сообщений и т.д.

Преимущества этого подхода на практике проявляются в упрощении модификации и тестирования системы. Например, смена базы данных или изменение интерфейса пользователя не затрагивает бизнес-логику системы, которая локализована в энтитях и используемых случаях.

Важно отметить, что чистая архитектура не является строгим набором правил, а скорее предоставляет общую схему, которую можно настроить и адаптировать в соответствии с специфическими требованиями вашего проекта.

ПРАКТИЧЕСКАЯ ЧАСТЬ

4.1. РАЗРАБОТКА СТРУКТУРЫ И ИНТЕРФЕЙСА ЛАБОРАТОРНОГО ПРОГРАММНОГО КОМПЛЕКСА

Программный комплекс представляет собой набор модулей, каждый из которых выполняет определенную функцию. Структура комплекса включает в себя:

1. Графический модуль, который отвечает за отображение оборудования в области с оборудованием, из которого мы собираем лабораторную установку, и непосредственно саму рабочую область, в которой пользователь сначала собирает лабораторную установку, после чего у него происходит сам процесс лабораторной работы. Графический модуль отображает все выходные параметры.
2. Физический модуль, в который поступает информация для расчёта физических параметров, после чего она возвращается в графический модуль.
3. Конфигурационный модуль, в котором происходит заполнение входных параметров. Пользователь заполняет JSON-файл с ресурсами на оборудование (ссылки или пути до картинок), устанавливает связи между ними для корректного сбора установки и прописывает физические формулы для расчёта.

Интерфейс программного комплекса должен быть интуитивно понятным и удобным для пользователя.

Интерфейс комплекса включает в себя следующие элементы:

1. Рабочая область, которая содержит основные элементы управления комплексом. К ним относятся кнопки для запуска модулей, поля для ввода данных, кнопки для расчёта параметров и кнопки для отображения результатов.
2. Область оборудования, которая содержит в себе оборудование для сбора рабочей установки, перетаскиванием можно переместить элемент оборудования в рабочую область.
3. Окно результатов, в котором отображаются результаты расчета. Окно результата содержит информацию, которая позволяет пользователю анализировать результаты.

Реализация структуры и интерфейса комплекса осуществляется с использованием современных технологий программирования. Для реализации структуры комплекса используются объектно-ориентированные методы программирования. Для реализации интерфейса комплекса используются технологии графического интерфейса пользователя.

Реализация комплекса осуществляется в несколько этапов:

1. Проектирование структуры комплекса, на этом этапе разрабатывается структура комплекса, которая включает в себя модули, функции модулей и связи между модулями.
2. Разработка интерфейса комплекса, на этом этапе разрабатывается интерфейс комплекса, который включает в себя элементы управления и элементы отображения результатов.
3. Реализация модулей комплекса, на этом этапе реализуются модули комплекса, которые выполняют функции, определённые на этапе проектирования.

4. Тестирование комплекса, на этом этапе проводится тестирование комплекса, которое позволяет выявить ошибки в структуре и интерфейсе комплекса.

В результате реализации комплекса должен быть получен программный продукт, который соответствует требованиям, предъявляемым к лабораторным программным комплексам.

4.2 ОСОБЕННОСТИ РЕАЛИЗАЦИИ ГРАФИЧЕСКОГО МОДУЛЯ.

Графический модуль является основным модулем комплекса. Он отвечает за отображение оборудования в области с оборудованием и за отображение рабочей области. Также модуль также отображает все выходные параметры.

Данный модуль реализован с использованием технологий графического интерфейса пользователя. Для отображения оборудования и рабочей области используются элементы управления, такие как кнопки, поля ввода, списки и т. д..

Графические элементы управления и отображения реализованы с использованием современных технологий программирования, таких как HTML, CSS и JavaScript.

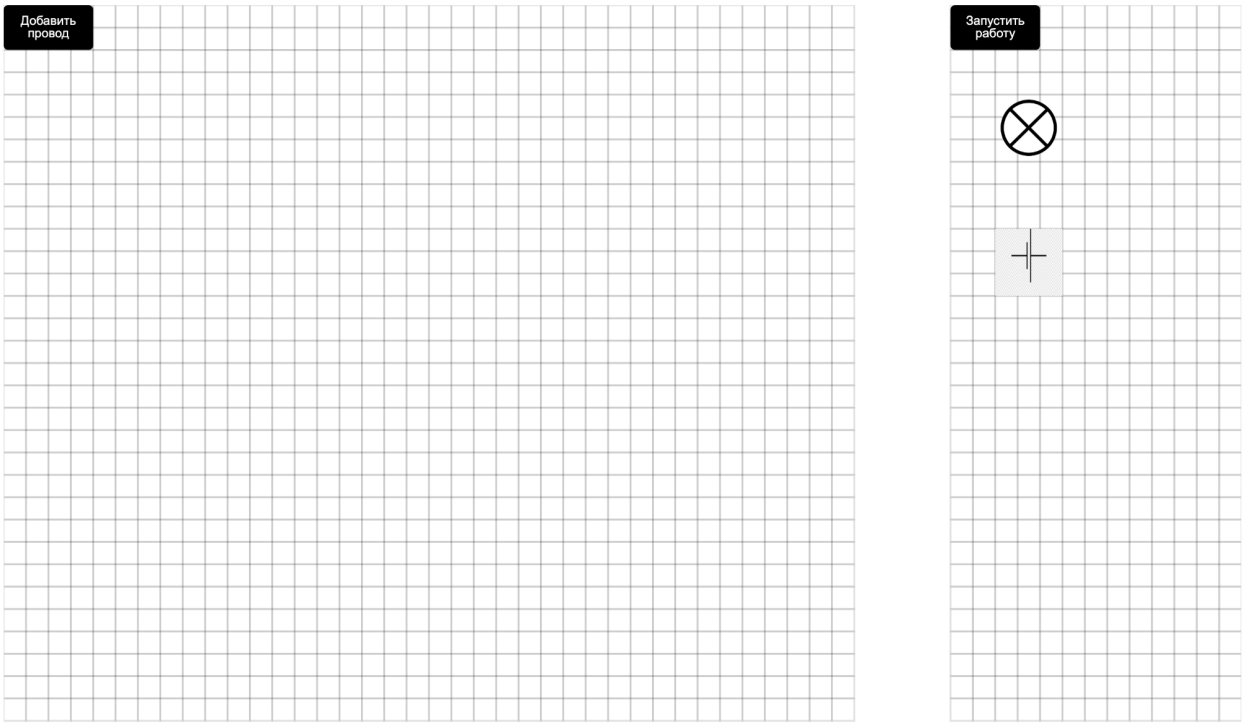

Рисунок 1 - Иллюстрация устройства графического модуля

4.3 ОСОБЕННОСТИ РЕАЛИЗАЦИИ ФИЗИЧЕСКОГО МОДУЛЯ.

Физический модуль является вспомогательным модулем комплекса. Он выполняет расчёт физических параметров на основе информации, поступающей из конфигурационного модуля. Также модуль возвращает результаты расчёта в графический модуль.

Данный модуль реализован с использованием математических методов и алгоритмов. Для расчёта физических параметров используются уравнения и формулы, которые описывают физические процессы.

Результаты расчёта физических параметров возвращаются в графический модуль в виде JSON-объекта.

```
1  import {PhysicsNotation} from '@labs/server';
2  import type {Inputs, Outputs, ProcessedPhysics} from './types';
3
4  const math = require('mathjs');
5
6  export const create = <T extends PhysicsNotation>(notation: T): ProcessedPhysics<T> => {
7      const values: Partial<Inputs<T>> = {};
8
9      const feed = (updateWith: Partial<Inputs<T>>) => {
10         Object.assign(values, updateWith);
11     };
12
13     const compute = (): Partial<Outputs<T>> => {
14         const targets = Object.entries(notation.produces);
15
16         const scope = Object.assign(notation.constants, notation.initial, values);
17
18         const result: Record<string, number | undefined> = targets.reduce((acc, curValue) => {
19             const [name, formula] = curValue;
20             try {
21                 return {...acc, [name]: math.evaluate(formula, scope) as number | undefined};
22             } catch {
23                 return {...acc, [name]: undefined};
24             }
25         }, {});
26
27         Object.assign(values, result);
28
29         return result as Outputs<T>;
30     };
31
32     return {feed, compute, values};
33 };
34
```

Рисунок 2 - Устройство физического модуля

1 строчка - Использование конфигурации физики из конфигурационного модуля.

4 строчка - Используем библиотеку mathjs для обработки всех вычислений. Встроенные в JavaScript математические вычисления основаны на float и хранятся в виде бинарного числа, из-за этого даже выражение 0.1 + 0.2 может равнятся 0.3 + 1e-7, mathjs решает эту проблему в дополнение она умеет производить математические вычисления, которые хранятся у нас в конфигурации.

9 строчка - Реализация функции передачи значений в компонент. В конфигурации прописана точка входа в лабораторную, откуда будет запускаться приложение. Далее в главе будет описан процесс расчета значений каждого компонента лабораторной работы.

18 строчка - Реализации функции расчета значений компонента. Используя константы компонента, значения, полученные от других компонентов, получаем выход компонента.

```
export type PhysicsNotation = {
    initial?: Record<string, number>;
    // Опциональный параметр, деталь (например батарейка)
    dependsOn?: string[];
    // В системе СИ
    constants: Record<string, number>;
    produces: Record<string, string>;
};
```

Рисунок 3 - Описание сущности физического элемента

initial - изначальное состояние компонента, например, первоначальное напряжение в батарее.

dependsOn - значение, от которого зависит компонент, названия других компонентов из конфигурации. Компонент может не иметь зависимостей (например батарея).

constants - константные значения компонента (в системе СИ)

produces - формулы каждого значения, которые производит компонент.

4.4 ОСОБЕННОСТИ РЕАЛИЗАЦИИ КОНФИГУРАЦИОННОГО МОДУЛЯ.

Конфигурационный модуль является основным модулем комплекса. Он отвечает за заполнение входных параметров, таких как ресурсы на оборудование, связи между оборудованием и физические формулы.

Конфигурационный модуль реализован с использованием JSON-формата. JSON-файл содержит ресурсы на оборудование в виде ссылок или путей до

картинок, связи между оборудованием в виде JSON-массива и физические формулы в виде JSON-объектов.

JSON-файл загружается в конфигурационный модуль при запуске комплекса. Конфигурационный модуль анализирует JSON-файл и заполняет входные параметры.

```
1  {
2    "cell": {
3      "width": 25,
4      "height": 25
5    },
6    "equipment": {
7      "lamp": {
8        "title": "Лампа",
9        "measure": [3, 3],
10       "ports": [
11         { "position": [0, 2] },
12         { "position": [4, 2] }
13       ],
14       "src": {
15         "base": "image1",
16         "active": "image2"
17       },
18       "physics": {
19         "dependsOf": ["U", "R"],
20         "produces": {
21           "I": "U / R"
22         }
23       }
24     },
25     "battery": {
26       "title": "Батарея",
27       "measure": [3, 3],
28       "ports": [
29         { "position": [0, 2] },
30         { "position": [4, 2] }
31       ],
32       "src": {
33         "base": "image"
34       },
```

Рисунок 4 - Устройство конфигурационного JSON файла (½)

```
18          "physics": {
19            "dependsOf": ["U", "R"],
20            "produces": {
21              "I": "U / R"
22            }
23          }
24        },
25        "battery": {
26          "title": "Батарея",
27          "measure": [3, 3],
28          "ports": [
29            { "position": [0, 2] },
30            { "position": [4, 2] }
31          ],
32          "src": {
33            "base": "image"
34          },
35          "physics": {
36            "initial": {
37              "U": 220,
38              "R": 4
39            },
40            "produces": {
41              "U": "U",
42              "R": "R"
43            }
44          }
45        }
46      },
47      "proofOfDone": {
48        "required": ["lamp", "battery"],
49        "connections": [["lamp", "battery", 2]]
50      },
51      "entry": "battery"
52    }
53
```

Рис. 5 - Устройство конфигурационного JSON файла (2/2)

1. Ключ cell отвечает за определение размера клетки в рабочей области. Это означает, что при работе с графическим интерфейсом или другими элементами, клетка будет использоваться как единица измерения для распределения пространства.

2. Ключ equipment описывает каждый элемент оборудования. Он содержит в себе следующие подключенные ключи:
   1. measure: ключ содержит информацию о размере устройства.
   2. ports: ключ, который определяет точки для подключения устройства.
   3. src: ключ, содержащий ссылку на изображение элемента оборудования.
   4. physics: ключ, который описывает физические характеристики устройства.
3. Устройство блока physics было описано ранее. Он позволяет дополнительно специфицировать физические свойства устройства для дальнейшего анализа или использования в системе.

ДЕМОНСТРАЦИЯ ФУНКЦИОНАЛА ПРОГРАММНОГО КОМПЛЕКСА

5.1 ИНТЕРФЕЙС ИНИЦИАЛИЗАЦИИ ПРОГРАММНОГО КОМПЛЕКСА

На рисунке 6, можно увидеть стартовый экран программного комплекса, на котором отображены все основные функции, а именно: в правой части приложения мы видим различное оборудование из которого студенты смогут собрать установку, в левой части мы видим рабочую область, в которую непосредственно переносится оборудование, под рабочей областью мы видим инструменты, кликая на которые, мы можем переключаться между режимами управления мышью, возможные режимы:

1. Знаком курсора обозначен режим, в котором можно двигать элементы
2. Знаком провода обозначен режим, в котором можно создавать провод в рабочей области
3. Знаком закругленного провода обозначен режим, в котором можно создавать закругленный провод
4. Знаком ластика обозначен режим, в котором мы можем удалять элемент с рабочей области
5. Знаком вопроса обозначен режим, который открывает модальное окно с справкой

Рис. 6 - Главный экран лабораторного комплекса

5.1 СБОРКА ЛАБОРАТОРНОЙ УСТАНОВКИ

На рисунке 7 показана лабораторная установка в собранном виде.

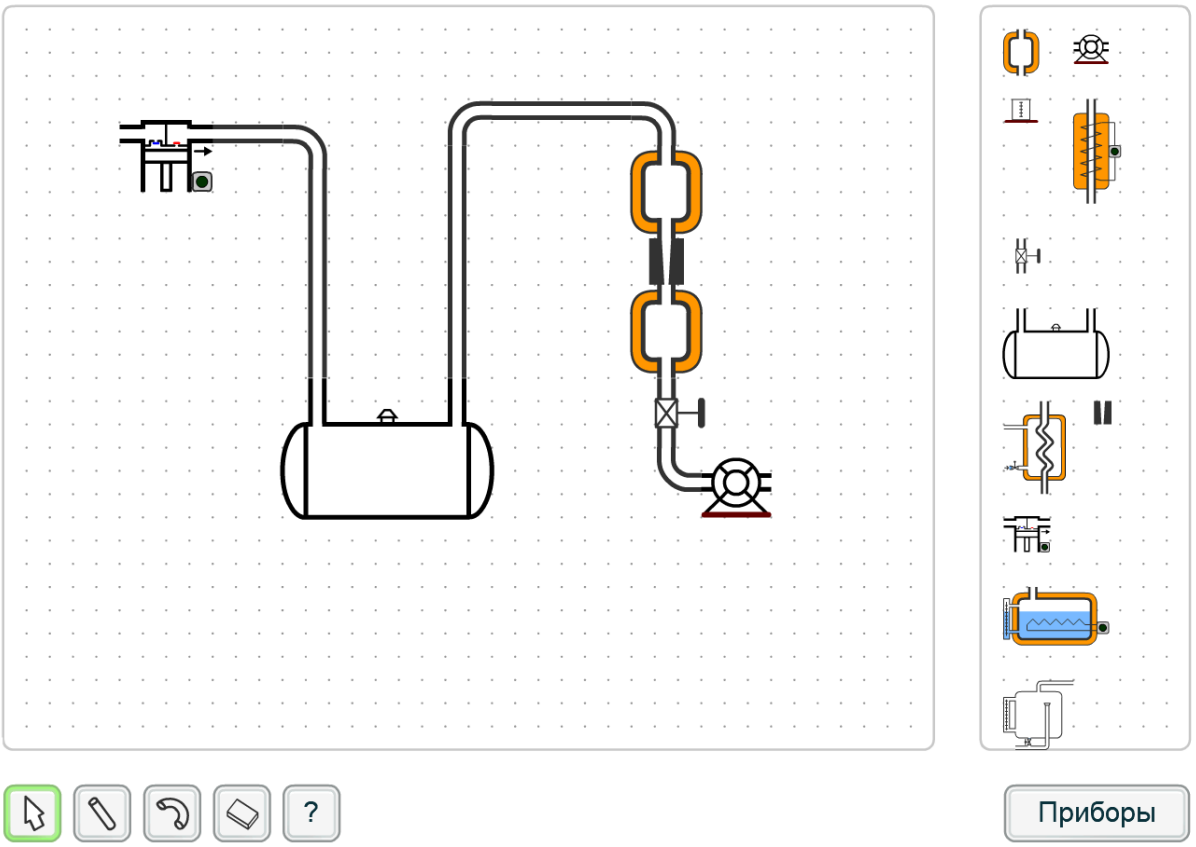

Рис. 7 - Собранная лабораторная установка

После того, как лабораторная установка будет собрана, необходимо нажать на кнопку “Приборы”, которая находится под панелью с оборудованием.

После чего студента перекидывает на панель с датчиками, которые необходимо установить, перетаскивая их на установку (Рисунок 8)

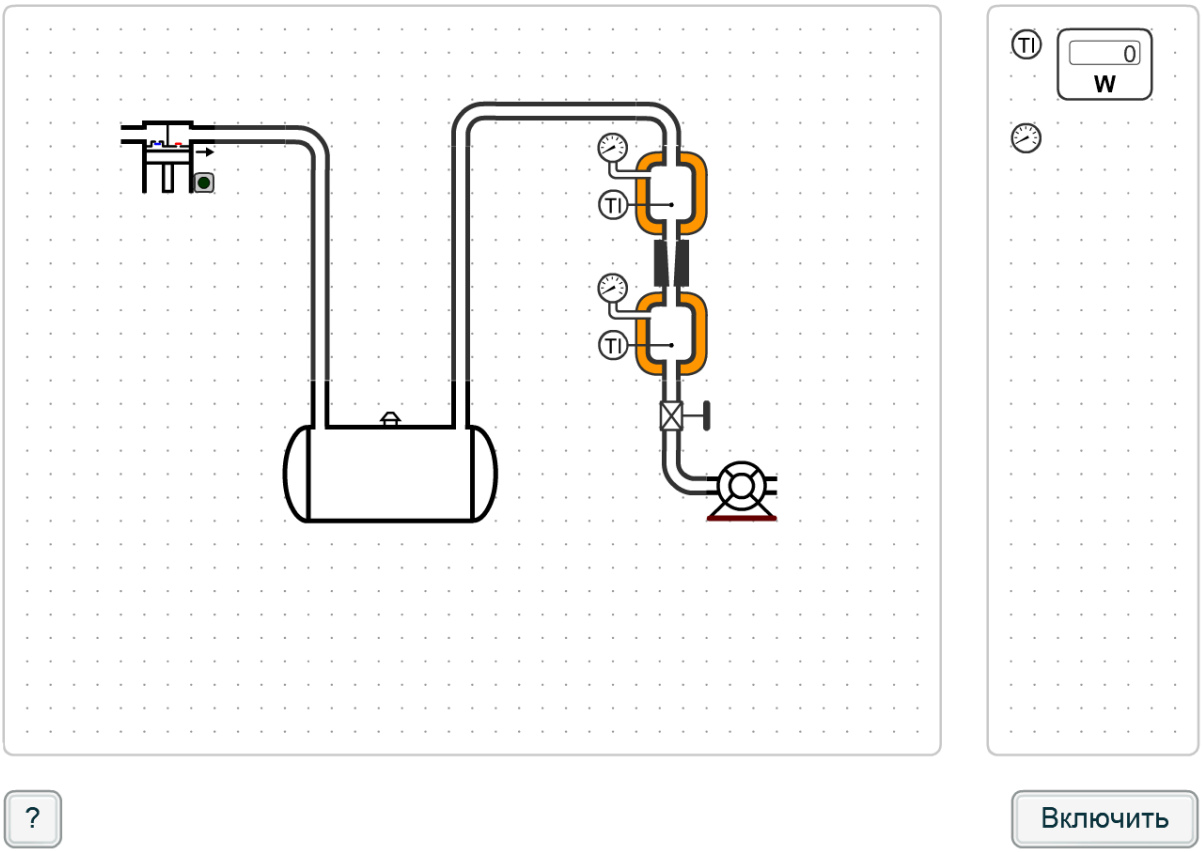

Рис. 8 - Панель с датчиками

5.2 ЗАПУСК ЛАБОРАТОРНОЙ УСТАНОВКИ

Для запуска лабораторной установки, необходимо нажать на кнопку “Включить”, после чего, в случае успешной сборки, необходимо нажать на кнопку на конкретной детали (Рисунок 9), после чего установка будет запущена (Рисунок 10).

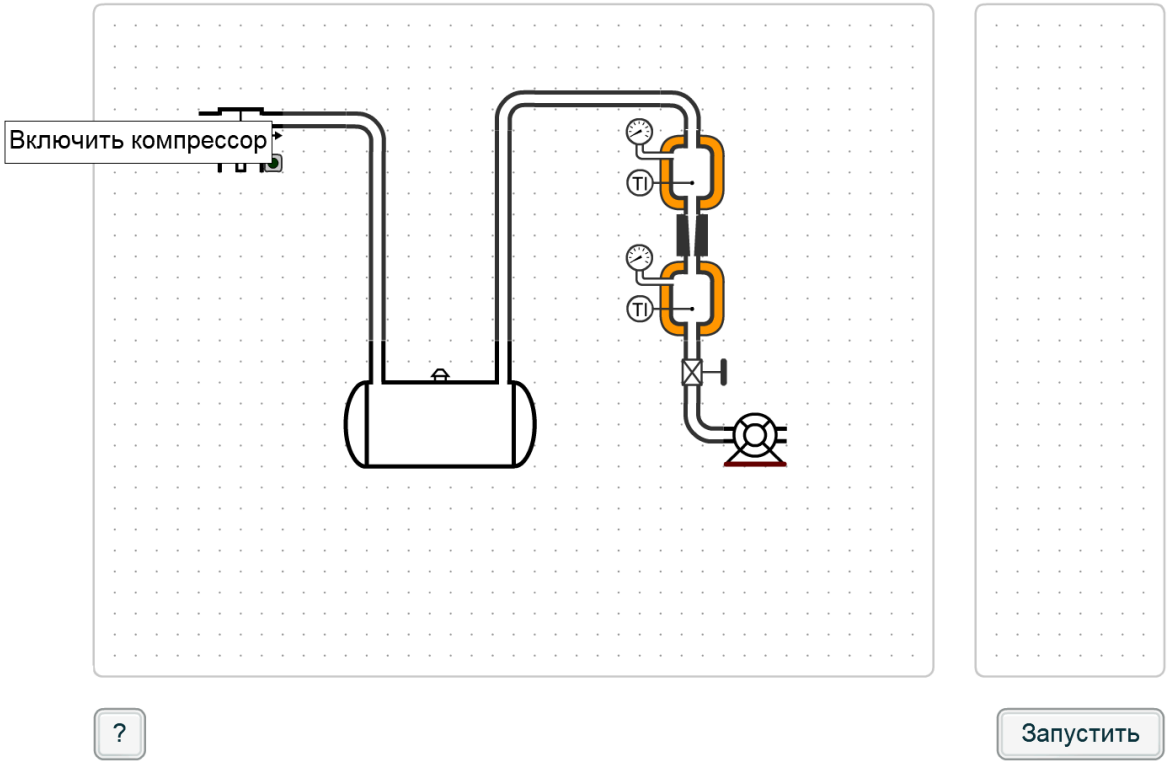

Рис. 9 - Установка готовая к запуску

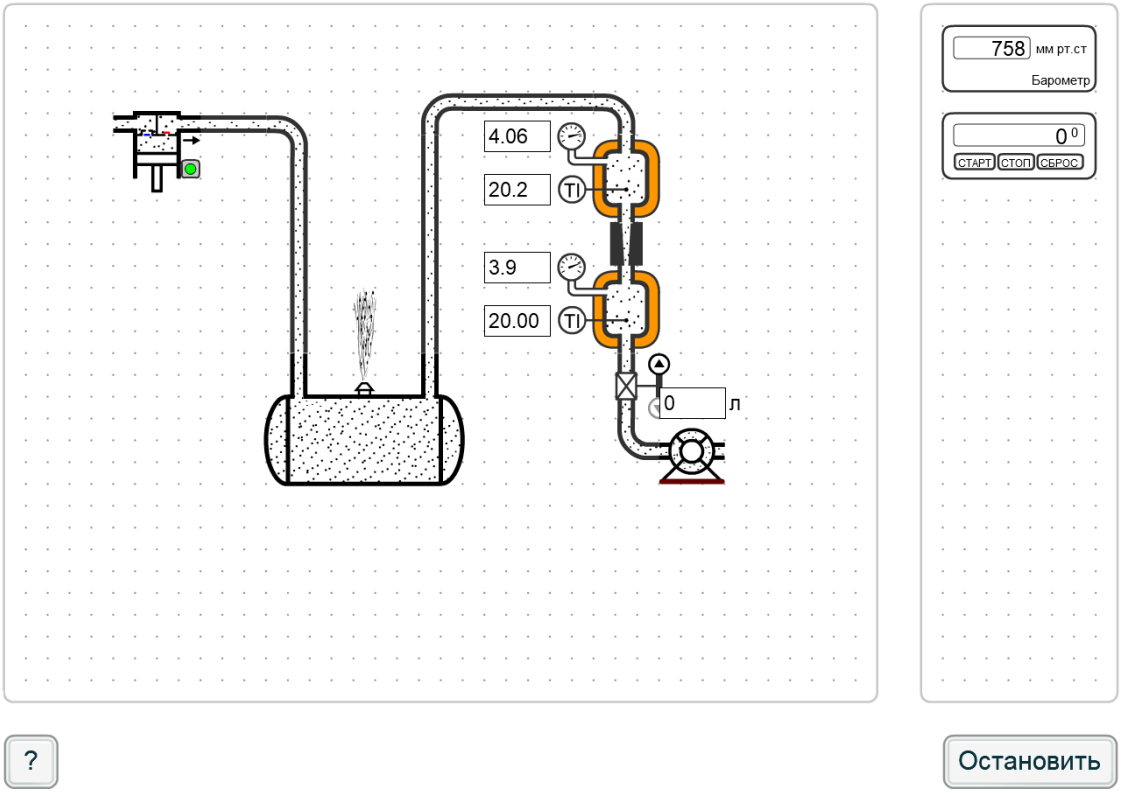

Рис. 10 - Запущенная установка

5.3 ОШИБКИ ВОЗМОЖНЫЕ ПРИ ЗАПУСКЕ

В случае если студент не использовал все необходимые детали для установки, при попытке запуска, выводится предупреждение (Рисунок 11)

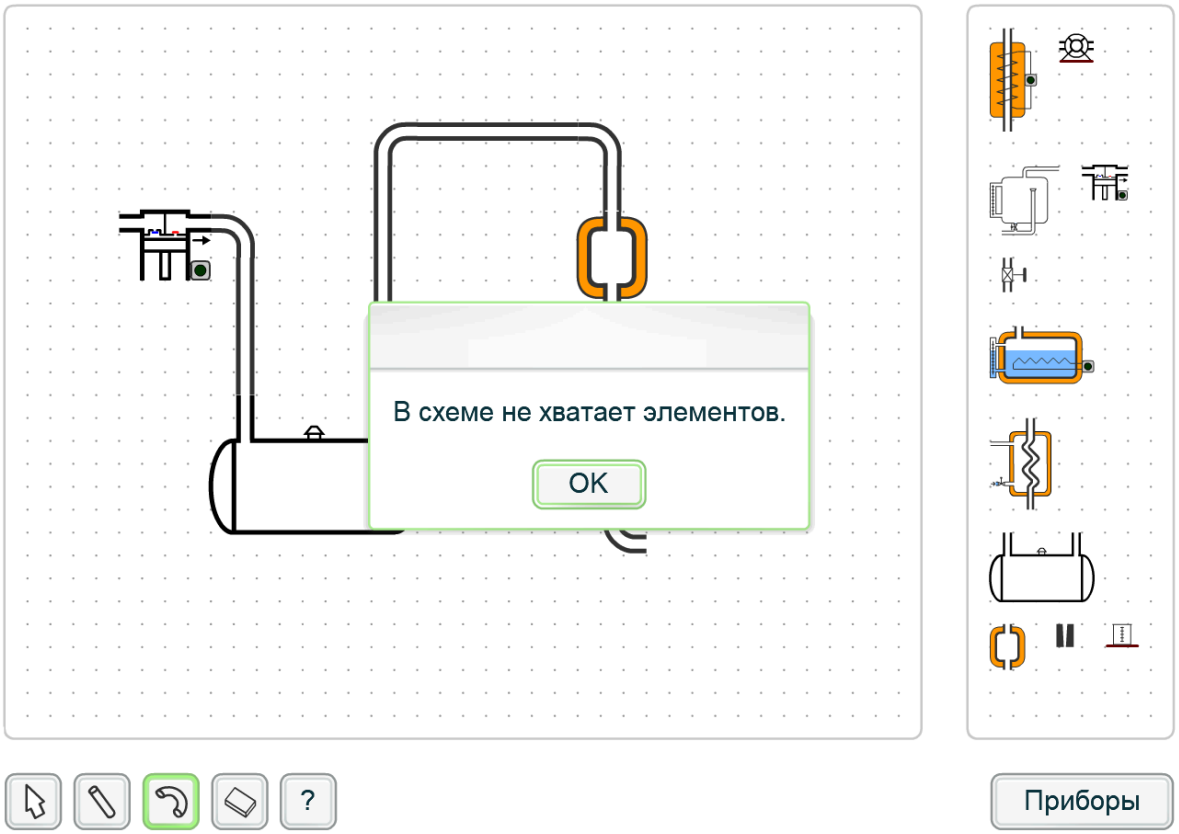

Рис. 11 - Предупреждение о неверной сборке

В случае, если студент не установил все датчики, ему выводится предупреждение о том, что он не установил все необходимые измерительные приборы (Рисунок 12)

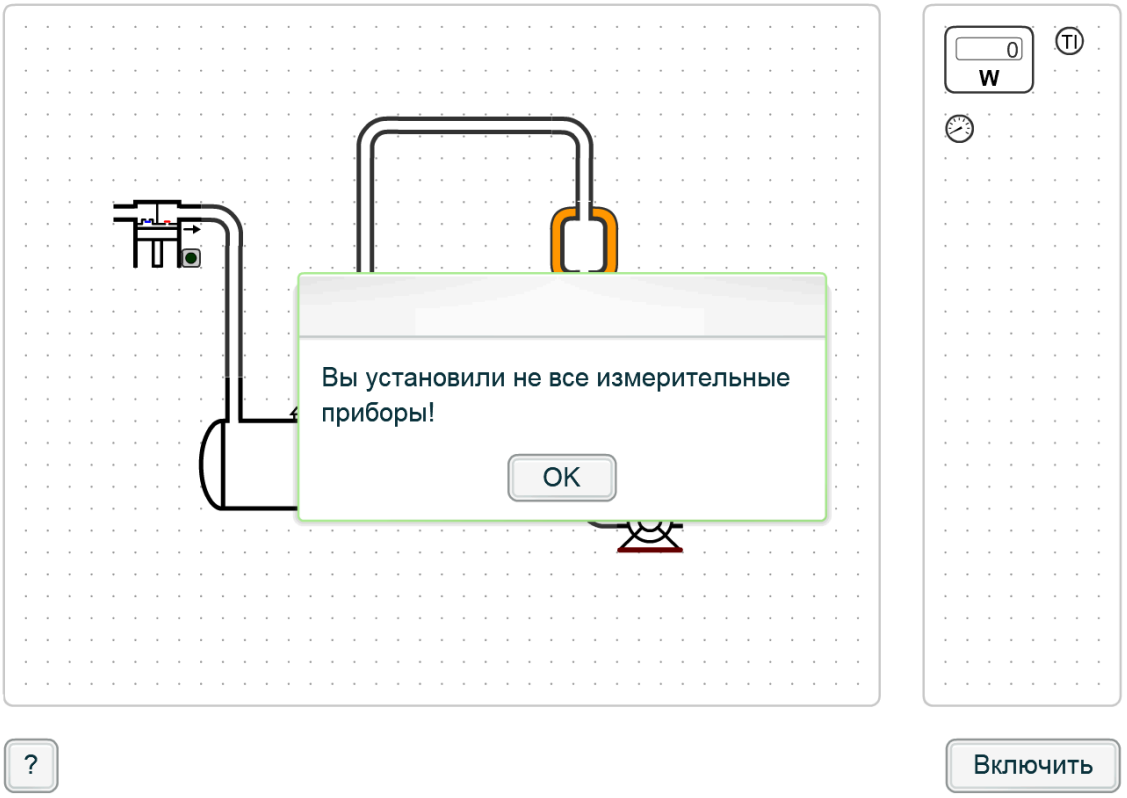

Рис. 12 - Предупреждение о том, что не установлены все измерительные приборы

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломной работы был разработан программный комплекс, ориентированный на визуализацию истечения идеального газа при дросселировании с целью обеспечить студентам возможность практического применения своих знаний и навыков в области теплофизики. Разработанный образовательный инструмент представляет собой удобное средство как для студентов, так и для преподавателей, позволяя студентам визуально осмыслить процессы, изучаемые в теории, и применить их на практике, а преподавателям - эффективно контролировать прогресс обучения.

Используя программный комплекс, студенты смогут лучше понять основные принципы и законы, связанные с процессами истечения идеального газа при дросселировании, и применить их для расчета и определения соответствующих параметров системы. Лабораторный комплекс также поможет студентам углубить свои знания и умения.

Также, преподаватели, получают возможность эффективно мониторить и оценивать уровень знаний и понимания студентов, а также контролировать качество обучения в динамике. Это помогает преподавателям адаптировать свой подход к обучению, учитывая особенности группы студентов, повышая эффективность учебного процесса и обеспечивая лучшее взаимодействие с учащимися.

Таким образом, разработанный программный комплекс является значимым инструментом для образовательного процесса в области теплофизики, способствуя улучшению качества обучения и обеспечивая студентам и преподавателям ценный ресурс для углубленного изучения процессов истечения идеального газа при дросселировании.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ТЕПЛОВЫЕ МАШИНЫ ЖД - ПАРОВЫЕ ТУРБИНЫ. URL: https://lokomo.ru/info/teplovye-mashiny-zhd/Page-17.html (дата обращения: 15.03.2024).
2. Энциклопедия по машиностроению XXL. URL: https://mash-xxl.info/info/235346/?ysclid=lx0qmmokhd761007671 (дата обращения: 15.03.2024).
3. Первый закон термодинамики. URL: https://studwood.net/2005167/matematika_himiya_fizika/rabota_izmeneniya_obyoma_obyomnoy_deformatsii_diagramma?ysclid=lx0qtj0tnm458142338 (дата обращения: 15.03.2024).
4. Сопло Лаваля — машина, создающая сверхзвук URL: https://naked-science.ru/article/nakedscience/soplo-lavalya?ysclid=lx0qvviqmy33719023 (дата обращения: 21.03.2024).
5. Истечение и дросселирование газов и паров. URL: https://infopedia.su/28xa252.html?ysclid=lx0qy1fm4v770784020 (дата обращения: 21.03.2024).
6. MDN документация по HTML и CSS. URL: https://developer.mozilla.org/ru/docs/Learn/CSS/First_steps/Getting_started (дата обращения: 05.05.2024).
7. TypeScript: язык для разработки больших приложений на JavaScript. URL: https://habr.com/ru/articles/153077/ (дата обращения: 06.05.2024).
8. Node js документация. URL: https://nodejs.org/docs/latest/api/ (дата обращения: 18.05.2024).
9. Nest js документация. URL: https://docs.nestjs.com/ (дата обращения: 18.05.2024).

10. Обход в ширину. URL: https://neerc.ifmo.ru/wiki/index.php?title=Обход_в_ширину (дата обращения: 19.05.2024).